C09 ScriptableObjects

CRE132 worksheet

What you'll make

Two kinds of slime that patrol left and right: slow blue ones and fast green ones. Their speed, patrol distance, health and colour don't live on the slimes. They live in two **data assets**, **Blue Slime** and **Green Slime**, and each slime points at one of them.

You'll write a **ScriptableObject**, a script that holds data as an asset in the Project window instead of being a component on a GameObject. Change one asset and every slime that uses it changes at once.

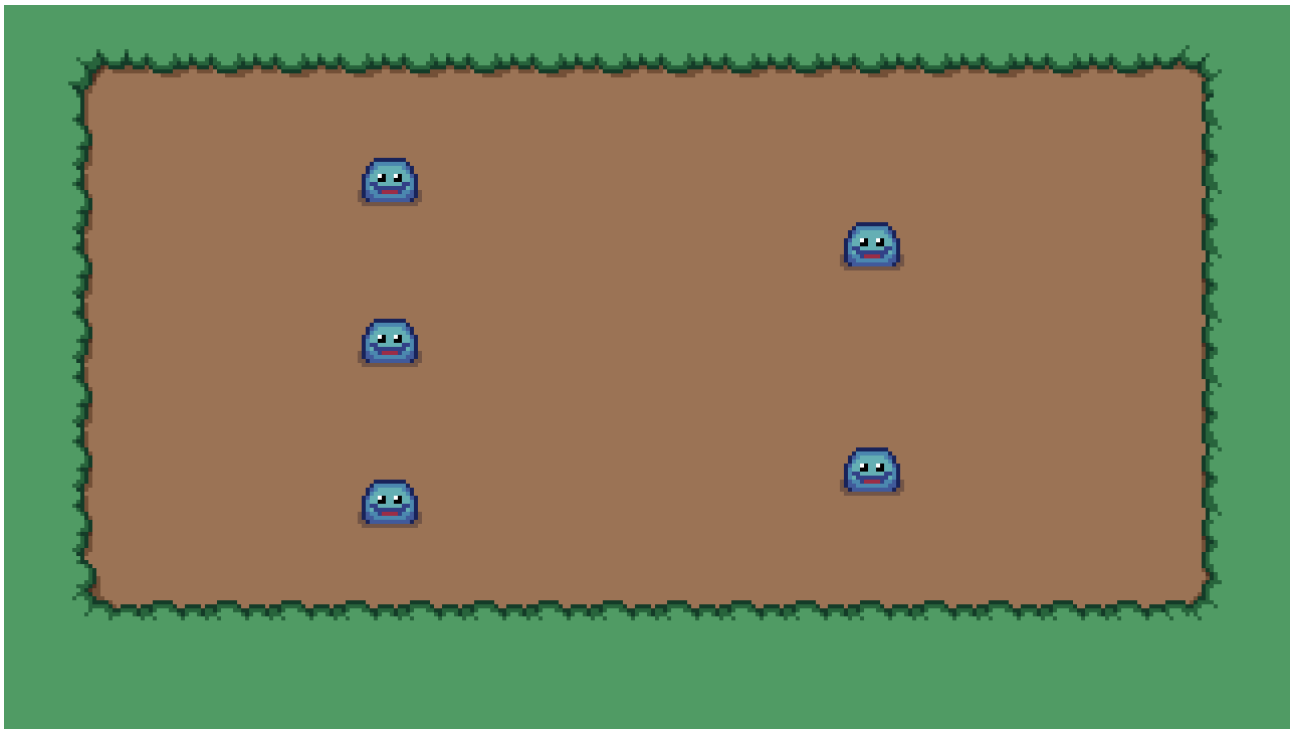


Figure 1: The finished scene: five slimes patrolling a dirt clearing, three on the left and two on the right.

Before you start

1. Unzip `CRE132_C09_ScriptableObjects_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C09_ScriptableObjects_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C09_ScriptableObjects`.
3. Press **Play**. The slimes bob (their C07 animation) but stay where they are. Press Play again to stop.

Inside **Slimes** in the Hierarchy are five instances of C08's **Slime** prefab: three called **Blue Slime** on the left and two called **Green Slime** on the right.

The problem: if speed, health and colour were fields on the patrol script, you'd set them by hand on every slime, and keep all the blue ones the same by hand too. A shared data asset fixes that.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The data: SlimeData

Make a folder `Assets/_game/Scripts/C09_ScriptableObjects` and create a script `SlimeData` in it (the MonoBehaviour Script menu is fine: you'll replace everything). Replace it with:

```
// A data asset that describes one kind of slime. Make one with Create > CRE132 > Slime
// Data.
// Many slimes can point at the same asset, so changing the asset changes all of them
// at once.
using UnityEngine;

[CreateAssetMenu(fileName = "New Slime Data", menuName = "CRE132/Slime Data")]
public class SlimeData : ScriptableObject
{
    // Public, because this class is only a container: other scripts read these values.
    public string displayName = "Slime";
    public int maxHealth = 3;
    public float moveSpeed = 1f;
    public float patrolDistance = 2f;
    public Color tint = Color.white;
}
```

- `: ScriptableObject` instead of `: MonoBehaviour`. A ScriptableObject is never attached to a GameObject; it lives in the Project window as an asset, like a sprite or a prefab.
- `[CreateAssetMenu]` adds **Create > CRE132 > Slime Data** to the Project window's right-click menu, so you can make assets from it.
- **Public fields** are fine here. SlimeData has no behaviour: it's only a container of values that other scripts read.

2. The data assets

Make a folder `Assets/_game/ScriptableObjects/Slimes`. Right-click in it and choose **Create > CRE132 > Slime Data**. Name it **Blue Slime**. Make a second one called **Green Slime**. Click each and fill in the Inspector:

Field	Blue Slime	Green Slime
Display Name	Blue Slime	Green Slime
Max Health	3	1
Move Speed	1	3
Patrol Distance	2.5	4
Tint	white (R 1, G 1, B 1)	yellow tint (R 1, G 1, B 0.45)

The yellow tint mixes with the blue sprite to make the green slimes green. For the colour, click the **Tint** box and type the values; if the picker shows 0 to 255, use 255, 255, 115.

3. Using the data: SlimePatrol

Create `SlimePatrol` in the same scripts folder:

```
// Moves a slime left and right around where it started.
// How fast, how far and what colour all come from a SlimeData asset, not from this
// script.
using UnityEngine;

public class SlimePatrol : MonoBehaviour
{
    [SerializeField] private SlimeData data;

    private SpriteRenderer spriteRenderer;
    private Vector3 startPosition;
    private int direction = 1;

    private void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        startPosition = transform.position;
        Debug.Log(data.displayName + " is ready with " + data.maxHealth + " health");
    }

    private void Update()
    {
        // Set the colour every frame so changes to the asset show up straight away in
        // Play mode.
        spriteRenderer.color = data.tint;

        Vector3 step = new Vector3(direction * data.moveSpeed * Time.deltaTime, 0f,
            0f);
        transform.position = transform.position + step;

        // Turn around at either end of the patrol.
        if (transform.position.x > startPosition.x + data.patrolDistance)
        {
            direction = -1;
        }
        else if (transform.position.x < startPosition.x - data.patrolDistance)
        {
            direction = 1;
        }
    }
}
```

- **`[SerializeField] private SlimeData data`** is a slot for a data asset. Everything the slime needs to know comes through it: `data.moveSpeed`, `data.patrolDistance`, `data.tint`.
- **`direction`** is 1 (moving right) or −1 (moving left). When the slime gets `patrolDistance` past its start, it turns round.
- **The colour is set every frame** in `Update`, so if you change the asset's Tint while playing, you see it straight away.
- **Nothing is copied.** Three slimes pointing at Blue Slime all read the same one asset.

Add it: select the three **Blue Slime** objects (click the first, Ctrl-click the others) and **Add Component > SlimePatrol**. Drag the **Blue Slime** asset from the Project window into the **Data** slot. Do the same for the two **Green Slime** objects with the **Green Slime** asset.

Test it: press Play. The blue slimes patrol slowly over a short distance; the green ones turn green and patrol fast and far. The Console shows a "... is ready with ... health" line for each slime.

4. Shared data, and a warning

While the game is playing, select the **Blue Slime** asset in the Project window and change **Move Speed** to 3, or its **Tint**. All three blue slimes change together.

Now stop playing and look at the asset again: **your change is still there**. Changes to a component in Play mode are thrown away when you stop (C04), but changes to an **asset** are kept. Set Move Speed back to **1** and the Tint back to white.

That's why the slime's *current* health, which goes down during a game, should never be stored in SlimeData. Values that change while playing belong in a MonoBehaviour; the asset holds the starting values.

Check your work

- ☐ **Create > CRE132 > Slime Data** appears in the Project window's right-click menu.
- ☐ Blue slimes patrol slowly (speed 1, distance 2.5); green ones are green and fast (speed 3, distance 4).
- ☐ The Console shows "Blue Slime is ready with 3 health" three times and "Green Slime is ready with 1 health" twice.
- ☐ Changing the Blue Slime asset in Play mode changes all three blue slimes.
- ☐ Compare with `CRE132_C09_ScriptableObjects_Finished` if something behaves differently.

Key points

- **ScriptableObjects are for data shared by many objects**: enemy types, item definitions, level settings.
- **Many objects point at one asset**; nothing is copied, so one change updates them all.
- **Asset changes made in Play mode stay** when you stop. Don't keep values that change during the game (like current health) in an asset.

If something goes wrong

- **There's no CRE132 menu under Create:** the `[CreateAssetMenu]` line is missing or misspelt, or the script has an error. Fix the first red error in the Console.
- **UnassignedReferenceException or NullReferenceException in SlimePatrol:** a slime's **Data** slot is empty. Check all five.
- **"The script needs to derive from MonoBehaviour" when you try to add SlimeData to an object:** that's expected. SlimeData isn't a component; make an asset from it instead.
- **The slimes don't move:** the asset's Move Speed is 0, or Patrol Distance is 0.
- **The green slimes look blue in Edit mode:** that's normal. The tint is applied when the game runs.

Try it

1. Make a **Red Slime** asset and give one of the slimes on the left to it.
2. Add a `float scale` to `SlimeData` and use it in `Start`: `transform.localScale = ...`
3. Look ahead: T09 uses ScriptableObjects for inventory items. What else in a game would you store this way?